

Cross-Plane Operational Provenance under Identifier Failure: A Reproducible Evaluation of EACP

Obede Bessa Rocha da Silva

Independent Researcher in Cloud Architecture and DevOps • Clermont, Florida, USA

Correspondence: obedebessa@gmail.com • ORCID: [0009-0001-0787-5710](https://orcid.org/0009-0001-0787-5710)

ABSTRACT

Cloud-native operations scatter audit-relevant evidence across delivery and runtime systems. This paper presents EACP 1.3, a domain-specific operational-provenance profile and materialized retrieval index for records separately emitted by source systems. It retains native evidence pointers, uses typed and scoped exact links, and abstains when the selected key is missing or ambiguous. A 30-seed adversarial campaign measures the cost of identifier failure: random missingness of 1%, 5%, 10%, and 20% reduced strict six-event exact-chain recovery to 94.17%, 73.25%, 53.00%, and 26.17%. A compound scenario yielded 47.33% exact recovery, 72.86% pairwise recall, and 19.01% abstention, with no observed false join under the declared synthetic invariants. A paired SQLite ablation isolated retrieval benefits and costs. At 100,000 events, removing the targeted lookup index increased warm p95 by 5.65 times for service queries and 73.91 times for correlation queries; the two indexes occupied 22.776% of the full database, and removing both reduced median bulk-ingest time by 17.90%. For two-emitter evidence, an earlier public run established the evidence shape. In a prospectively committed 3-by-3 cohort across Kubernetes v1.34.8, v1.35.5, and v1.36.1, 0/9 first attempts satisfied all predeclared criteria because a lifecycle-invalid assertion requested artifact metadata before upload; every failure was retained. After a narrow direct-child amendment, 9/9 confirmatory first attempts satisfied all predeclared criteria, 3/3 per version. Each successful run retained a present no-ID control that remained unjoined, labeled the target-bound HTTP 403 as adapter-explicit, checked the OCI digest separately, and passed TAR-parity and attestation verification. The GitHub build-provenance attestation names only the in-run TAR as its subject; completed-state finalization is a separate checksum-bound, publicly cross-checkable observation. These descriptive results support bounded feasibility, explicit failure behavior, and controlled compatibility—not causal inference, source truth, production reliability, independent replication, or distributed-scale performance.

Keywords—*cloud-native systems; operational provenance; control plane; auditability; DevOps; evidence index; reproducibility*

1 Introduction

Cloud-native infrastructure is governed by several control planes at once. A deployment controller reconciles desired and observed state; an identity system decides whether an actor may change a resource; a policy engine permits or rejects a workload; a telemetry pipeline reports health; and incident and recovery systems record remedial action. Kubernetes explicitly models controllers as loops that move current state toward desired state [1], and its audit facility produces chronological, security-relevant records about cluster activity [2]. Yet an operator explaining why a service reached a particular state may still need to traverse multiple tools, identifiers, schemas, retention policies, and clocks.

This fragmentation matters for both operations and assurance. NIST SP 800-53 includes event logging, audit-record generation and protection, configuration-change control, continuous monitoring, incident handling, and contingency-planning controls [3]. The Cybersecurity Framework 2.0 organizes outcomes across Govern, Identify, Protect, Detect, Respond, and Recover [4]. These sources establish the need for accountable operational evidence, but they do not prescribe a small runtime index that binds a deployment, identity decision, policy result, alert, incident action, and recovery artifact at service granularity.

Existing specifications and systems solve adjacent problems. CloudEvents defines a vendor-neutral envelope and contextual attributes for event data [5]. OpenTelemetry defines a common log data model and correlation fields [6]. W3C PROV supplies a general model for entities, activities, and agents [7], [8]. Distributed tracing, whole-system provenance, service-provenance systems, software-supply-chain attestations, and component-metadata stores cover substantial parts of the same design space [9]–[12], [27]–[38]. EACP does not replace those mechanisms and does not claim a new general provenance model. It is evaluated here as a domain-specific operational-provenance profile and materialized retrieval index for records separately emitted by delivery and runtime-control systems.

The central research question is: to what extent can a lightweight, asynchronous evidence-aware control plane correlate heterogeneous operational events without silently inventing linkage when evidence is incomplete or ambiguous? Five subquestions guide the evaluation: (RQ1) can the v1.2 projection reproduce known chains when identifiers are complete; (RQ2) how do exact reconstruction, join precision, recall, abstention, and evidence availability change under controlled missingness, identifier error, collision, duplication, delay, reordering, and clock skew; (RQ3) what processing, storage, and indexed-query costs arise; (RQ4) can GitHub Actions delivery metadata and Kubernetes API audit records emitted during the same execution attempt be joined by exact scoped correlation, with the declared OCI subject digest checked separately and a correlation-free negative control rejected, and what fraction of separately triggered first attempts satisfies fixed criteria when the Kubernetes minor version is varied across a balanced, predeclared cohort; and (RQ5) can a fixed OpenTelemetry Collector pipeline preserve the Kubernetes corpus needed by an external validator?

The paper contributes:

- **A conservative operational-provenance profile.** EACP 1.3 makes source identity explicit, scopes service identity, separates actor roles, represents multiple typed links, and retains source pointers and optional source digests without claiming source truth or causality.
- **Safe link resolution.** The reference resolver uses exact typed and scoped keys and abstains on missing or multivalued linkage; a lossless migration preserves the complete v1.2 13-column projection without upgrading legacy hashes into source authentication claims.
- **An off-path architecture.** Adapters, normalization, an append-only index, and a query API operate asynchronously by default and do not become a dependency of application requests.
- **A real two-emitter evaluation.** A GitHub-hosted workflow creates an audited kind cluster, deploys a digest-pinned workload, records an authorization denial against the exact positive target, and retains a correlation-free negative control. A first-attempt cross-version protocol preserves failures, and a separately committed amendment tests the narrowly corrected procedure without pooling generations.
- **An adversarial linkage study.** A deterministic campaign scores strict, correlation-only, and temporal policies against hidden chain truth across a declared frozen fault matrix and publishes every trial row, summary, figure, environment record, and checksum.
- **Continuity with the v1.2 evidence.** The original indexed SQLite cost study, Kubernetes audit corpus, and bounded OpenTelemetry preservation exercise remain reported with their original interpretation limits.

CLAIM BOUNDARY

The artifact demonstrates exact-link retrieval and detectable abstention for declared laboratory policies, plus delivery-to-runtime composition in one earlier three-attempt execution and two separately reported nine-run cohorts using fresh ephemeral single-node kind clusters. The first cohort's failures remain failures; the confirmatory cohort evaluates only the amended procedure. It does not infer causality, establish that a source told the truth, make source artifacts tamper-proof, or measure production-scale, managed-control-plane, human-investigation, concurrent-write, or application-request overhead.

2 Background and Related Work

2.1 Control planes, desired state, and audit records

Container orchestration illustrates why operational state is distributed. Kubernetes objects are records of intent, and controllers continually reconcile actual state toward that intent [1]. Its architecture reflects lessons from Borg and Omega about declarative configuration, reconciliation, and shared-state coordination [13]. Kubernetes auditing separately records who acted, what occurred, when it occurred, and which resource was affected [2]. Those records are valuable, but a production service can also be

altered by external delivery systems, cloud identity providers, infrastructure APIs, policy engines, and recovery procedures. A cluster audit log therefore represents one evidence plane, not the entire operational history.

2.2 Event normalization and observability

CloudEvents addresses interoperability by standardizing event context such as id, source, type, time, and data schema [5]. OpenTelemetry addresses the representation and transport of logs, traces, and metrics; its stable log data model includes source and observed timestamps, resource attributes, event name, and optional trace context [6]. OpenTelemetry's CI/CD semantic conventions additionally define pipeline-run and VCS attributes, although that convention was Release Candidate at the time of this study [33]. These standards inform EACP adapters and time semantics. EACP adds a scoped service identity, explicit actor roles, typed linkage, and source-evidence pointers for the narrower retrieval question: which records, separately emitted by their source systems, are explicitly associated with this operational transition?

The OpenTelemetry Collector is a vendor-neutral implementation for receiving, processing, and exporting telemetry [20]. Version 0.158.0 of the Contrib distribution provides a filelog receiver that can parse JSON records and a file exporter that serializes OTLP data [21], [22]. This study uses that Collector configuration as an existing reference pipeline for the shared tasks of reading, parsing, preserving, and exporting audit events. It is not treated as a functionally equivalent provenance database: the file exporter is not an indexed query store, and EACP-specific hashes and reconstruction indexes fall outside the common projection.

2.3 Provenance and distributed tracing

W3C PROV defines interoperable provenance in terms of entities, activities, agents, and their relationships [7], [8]. EACP can be projected onto that model: the actor is an agent, the operational action is an activity, and the source artifact is an entity. EACP is therefore a domain profile and retrieval materialization, not a replacement serialization or more expressive provenance model. SPADE, CamFlow, PASS, and Linux Provenance Modules capture and query deeper host or distributed dependency graphs [11], [12], [34], [35]. ProvenanceLens addresses service provenance in cloud systems [37], and KubeProvenance reconstructs Kubernetes custom-resource history from audit records [38]. Those systems narrow any defensible novelty claim: EACP contributes an evaluated composition of delivery and runtime-control evidence at service granularity.

Dapper and Pivot Tracing demonstrate that correlation identifiers and dynamic causal queries can make distributed request behavior observable with controlled overhead [9], [10]. Their principal unit is a request or runtime causal path. EACP instead targets control-plane and operational transitions that may happen minutes or days apart and may have no shared trace span. PASS captures storage provenance

automatically [11], and Linux Provenance Modules protect whole-system provenance [12]. Those approaches provide deeper lineage than EACP. The EACP trade-off is coarser service-level semantics, lower integration ambition, and retained pointers rather than replication of all source data.

2.4 Software-supply-chain provenance and attestations

in-toto models and verifies steps in a software supply chain and explicitly includes inspection, packaging, and deployment activities [27], [28]. Its Attestation Framework also defines a runtime-trace predicate [42]. SLSA 1.2 standardizes provenance and attestation expectations while allowing additional predicate types [29]. Sigstore and Cosign provide signature, identity, and transparency-log mechanisms for artifacts and attestations [30]. Tekton Chains observes completed TaskRuns and PipelineRuns, snapshots them, converts them into provenance formats, signs them, and stores them [31]. Grafeas provides a central API and reference implementation for component metadata, including build, deployment, vulnerability, and attestation occurrences [32].

These mechanisms overlap EACP substantially; they cannot be dismissed as build-only. The precise boundary is narrower. EACP treats signed attestations and component metadata as possible source evidence while materializing a service-oriented operational view that also admits authorization, policy, incident, and recovery records. Its prototype query store does not add cryptographic authenticity to those sources. Conversely, a verified supply-chain attestation need not prescribe the same cross-source service-lifecycle query. The live experiment therefore records both explicit operational correlation and an OCI digest rather than treating either as a universal causal key.

SIEM and log-management infrastructures can centralize collection, normalization, storage, and analysis across heterogeneous sources [26]. Their service model, source-pointer handling, reconstruction rules, persistence guarantees, and request-path relationship are product- and deployment-specific, so Table 1 does not assign uniform capabilities to the entire product category.

EACP does not claim a new hash function, event envelope, provenance graph, attestation format, or correlation primitive. Its contribution is the restricted profile, safe-resolution policy, materialized retrieval index, and failure-preserving evaluation of delivery-to-runtime composition defined here. Table 1 compares native subjects, linkage, integrity, and storage responsibilities without using capability checkmarks that would obscure extensibility.

Table 1. Native emphasis and precise boundary of EACP and adjacent mechanisms

Mechanism	Native subject / horizon	Native linkage	Integrity / authenticity	Query store	Relationship to EACP
OpenTelemetry logs + CI/CD [6], [33]	Telemetry and pipeline signals	Trace/span IDs; run and VCS attributes	Transport and deployment dependent	No durable evidence store prescribed	Candidate adapter input; native OTLP ingestion is not implemented here.
W3C PROV [7], [8]	Arbitrary entities, activities, and agents	Rich explicit provenance graph	None built in	No implementation prescribed	EACP fields are mappable to PROV; no exporter is evaluated.
in-toto + SLSA [27]–[29]	Supply-chain steps, artifacts, deployment, and extensible predicates	Layouts, links, builder inputs, outputs, and digests	Signed attestations; assurance depends on policy and level	Distribution and lookup remain implementation dependent	Potential authoritative delivery evidence; broader operational retrieval differs.
Sigstore / Cosign [30]	Artifacts, blobs, and attestations	Identity, signature, subject digest, log inclusion	Cryptographic verification under a trust policy	Registry and transparency log	Assurance layer for source attestations or signed snapshots.
Tekton Chains [31]	Completed TaskRun/PipelineRun and products	Run snapshot, materials, and products	Signs with configured key/KMS and format	Multiple configurable stores	A pipeline-plane emitter with close architectural overlap.
Grafeas [32]	Software-component metadata and occurrences	Notes/occurrences keyed to resources	Implementation dependent	Yes: metadata API/reference server	Closest metadata-store overlap; deployment occurrences are supported, but Grafeas does not prescribe this exact service-transition profile or resolver.
Whole-system/ service provenance [11], [12], [34]–[37]	Process, file, dataflow, host, distributed, or service lineage	Dependency or provenance graphs	Varies by system	Provided by implementations	Deeper lineage; EACP uses coarser native control-plane records.
Kubernetes audit + KubeProvenance [2], [38]	API requests/stages and resource history	auditID and object history	Audit-backend dependent	Audit backend; KubeProvenance adds queries	One runtime-control plane, not a delivery-to-runtime composition alone.
EACP 1.3	Service transitions across delivery and runtime operations	Typed, scoped exact links with abstention	Append-only prototype; optional source digest; no source authentication	Materialized retrieval index	Evaluated composition and bounded retrieval policy.

2.5 Relationship to institutional governance research

The author's prior MGRDI article proposes an institutional governance and maturity model for cloud and DevOps adoption [19]. The present study addresses a different research object and method: it defines a systems artifact, implements executable data structures and invariants, and measures a technical benchmark. It does not reuse the MGRDI seven-layer structure, maturity scale, diagnostic lenses, executive indicators, or implementation roadmap. The separation is important: institutional governance can motivate the need for evidence, but it does not establish the performance or correctness of an evidence index.

3 Research Method and Design Requirements

The study follows a design-science orientation: identify an operational problem, derive objectives from authoritative and technical literature, construct an artifact, demonstrate it, and evaluate measurable properties [14], [15]. This version remains a laboratory technical evaluation rather than a field evaluation. The artifact is judged against explicit requirements, a deterministic correlation-fault campaign, a reproducible SQLite microbenchmark and index ablation, an earlier three-attempt public GitHub Actions run, an initial predeclared nine-run cross-version cohort, a separately committed nine-run confirmatory cohort, the previously published Kubernetes corpus, and a bounded OpenTelemetry preservation exercise. Each hosted execution creates a fresh ephemeral Kubernetes control plane. Organizational usefulness, operator effort, production reliability, and managed-control-plane behavior remain future work.

Table 2. Design requirements and their operationalization

ID	Requirement	Operationalization in the reference artifact
R1	Heterogeneous source input	Synthetic six-plane data test breadth; GitHub Actions and Kubernetes emit separate record sets within one orchestrated execution attempt.
R2	Source authority	The index stores a source pointer and optional precisely scoped digest; native systems remain authoritative.
R3	Scoped service identity	Identity is compared as service type, scope type, scope ID, and local ID—not as an unscoped name.
R4	Typed, multivalued linkage	Records can retain operational correlation, workflow run, VCS revision, artifact digest, deployment UID, and other typed links.
R5	Safe resolution	Exact typed and scoped resolution returns matches or explicitly abstains as missing or ambiguous; inference is opt-in.
R6	Idempotent append-only index	A unique source key rejects replayed insertion; SQLite triggers reject ordinary UPDATE and DELETE.
R7	Off-path operation	Evidence processing remains separate from the application request path.
R8	Precise integrity boundary	Tuple and source-representation digests, attested in-run TARs, and locally finalized state are labeled separately; none is described as a truth guarantee.
R9	Measured robustness	Twenty-five declared fault scenarios score exact-chain accuracy, pairwise precision/recall, abstention, ambiguity, and availability.

ID	Requirement	Operationalization in the reference artifact
R10	Measured cost and index value	Ingest, storage, warm/cold query latency, equivalence, and index ablations are recorded at three volumes.
R11	Executable cross-plane fit	GitHub-hosted jobs use digest-pinned kind node images, checksum-verified kubectl binaries, and exact API-server/kubelet version assertions, plus a digest-pinned deployment, no-ID control, and expected RBAC denial.
R12	Bounded interoperability	EACP and OpenTelemetry Collector 0.158.0 consume identical captured Kubernetes bytes; an external validator checks preservation.
R13	Externally inspectable artifact	Inputs, every trial row, schemas, predeclared plan and amendment, all 18 selected first-attempt outcomes, public run metadata, checksums, pinned dependencies, tests, and claim boundaries accompany the paper.

The requirements are intentionally testable but incomplete. Safe abstention trades recall for protection against observable ambiguity and cannot detect every semantically wrong but internally consistent identifier. Append-only behavior is confined to the SQLite application boundary. Off-path placement is a design property, not a measured application-latency result. The hosted executions use one runner and one fresh ephemeral single-node cluster apiece; they remain one-provider, one-repository procedural repetitions. The preservation exercise covers only the functions shared by both pipelines. None of these requirements establishes field effectiveness, source veracity, legal admissibility, or universal performance superiority.

4 EACP Architecture and Evidence Model

Figure 1 separates three concerns. Source planes retain authoritative records and expose native events. The evidence path maps those records to a common service-level index, correlates explicit identifiers, and exposes reconstruction queries. The application data path remains unchanged by default. Selective inline enforcement could be added for high-risk actions, but it is outside this artifact and would change the availability and latency analysis.

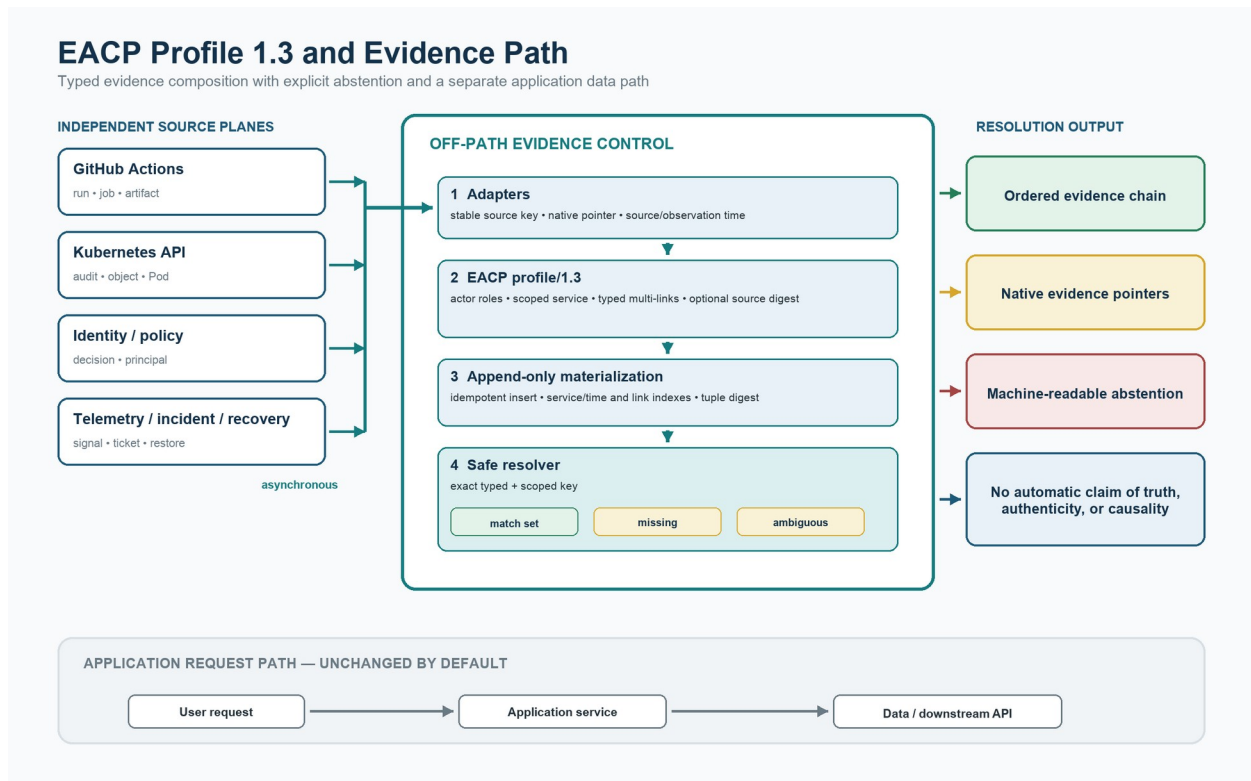


Figure 1. EACP observes heterogeneous source planes asynchronously, retains source pointers, and exposes operational reconstructions without becoming an application request-path dependency by default.

4.1 Components

- **Source adapters.** Adapters preserve stable source identity, native and observation time, evidenced actor roles, scoped service identity, source pointers, and every supported typed link. Missing context remains missing.
- **Profile validator and migrator.** JSON Schema and dependency-free semantic checks reject unknown or malformed fields. Migration preserves every value in the published 13-column v1.2 projection without recasting its legacy content hash as source authentication.
- **Safe link resolver.** Resolution uses exact typed and scoped keys. Missing and multivalued seed keys produce an explicit abstention; inferred links are excluded unless the caller opts in with a minimum declared confidence.
- **Append-only evidence index.** The index supports idempotent insertion, service/time and typed-link retrieval, plus a digest of its own canonical tuple. Retention, access control, and external digest protection remain deployment responsibilities.
- **Query and evidence-package API.** The API returns an ordered reconstruction, resolution status, and native evidence pointers. It never upgrades pointer existence into completeness, reachability, authenticity, or causality.

4.2 Tuple semantics

For an evidence-relevant event, EACP stores:

$$E = \{v, y_s, i_s, t_s, t_o, A, S, I, P, X, O, U, D?, L, Z?\}$$

Here v is the profile identifier; y_s and i_s are source type and source ID; t_s and t_o are source and observation timestamps; A is a possibly empty map of evidenced actor roles; S is a typed service identity with a typed scope; I , P , X , and O are intent, policy, action, and outcome; U is a source pointer; D is an optional digest whose covered representation and canonicalization are declared; L is a set of typed, scoped relationship assertions; and Z is a namespaced extension object. The ordered pair (y_s, i_s) is therefore part of the formal tuple and is the unique source key. This corrects the v1.2 paper, whose equation omitted those fields even though its database and invariants used them.

Actor roles distinguish initiator, triggering actor, execution principal, and attester without filling unsupported roles. A complete service key is $(\text{service.type}, \text{service.scope.type}, \text{service.scope.id}, \text{service.id})$. A link records type, value, scope, and evidence method: source-native, explicit, digest-match, or inferred. Only inferred links carry a confidence score, which is a declared score rather than an automatically calibrated probability. Multiple links preserve the fact that workflow run, revision, artifact digest, deployment UID, policy decision, trace, incident, and recovery identifiers answer different questions.

FIPS 180-4 defines SHA-256 for generating message digests [17]. In this profile a digest can reveal disagreement with a previously protected representation; it does not authenticate a producer, establish trusted time, or prove truth or causation. The tuple remains semantically compact because it indexes large artifacts instead of copying them. Physical compactness is separate: row headers, digest encoding, uniqueness constraints, and indexes can consume more bytes than the source rows, which the experiments measure.

4.3 Invariants and query classes

The 1.3 profile enforces: (I1) source-key uniqueness; (I2) offset-bearing timestamps without silent clock rewriting; (I3) complete scoped service comparison; (I4) no duplicate typed-and-scoped link key inside a record; (I5) confidence only and always for inferred links; and (I6) fail-closed structural validation. The SQLite materialization additionally rejects ordinary update and deletion. A strict resolution request selects exactly one eligible typed key from the seed, otherwise returning missing or ambiguous with no matches. Candidate records that carry a conflicting value in the selected type and scope are excluded and reported, never used as bridges. This controls an observable failure mode; it cannot expose an internally consistent but false assertion.

$$k(\ell) = \langle type, scope.type, scope.id, value \rangle \quad R(C, e, type) \in \{exact\ match\ set, \perp_{missing}, \perp_{ambiguous}\}$$

The resolver returns an exact match set only when seed e supplies one eligible key k for the requested type and each returned record carries that same key without a conflicting value in the selected type and scope. The two bottom values are machine-readable abstentions, not empty successful chains. Application-specific temporal or structural checks may add further abstention, as in the synthetic robustness experiment, but are not silently generalized into the core profile.

5 Threat, Privacy, and Failure Model

EACP assumes heterogeneous tools may omit context, clocks may differ, privileged operators may perform urgent manual changes, and source artifacts may expire. It does not assume that a database trigger defeats a privileged database administrator, that SHA-256 alone authenticates a record, or that a correlation identifier proves causality. The evidence index can itself reveal identities, services, incident timing, recovery objectives, and sensitive source locations. Consequently, evidence collection creates a new security and privacy asset rather than eliminating governance work.

Table 3. Primary threats, mitigations, and residual limitations

Threat or failure	Design response	Residual limitation
Duplicate delivery	Unique source key and transactional insertion	Incorrect or unstable source identifiers can still collide or fragment one event.
Tuple alteration	Append-only triggers and canonical SHA-256 digest	A privileged administrator can bypass application controls; no signature or external ledger is provided.
Missing source artifact	Persistent pointer rather than copied payload	The pointer can expire, authorization can change, and the target may be deleted.
Clock skew	Separate source and observed timestamps; skew injected independently of arrival	No clock correction is implemented; a conservative cadence guard can abstain on valid but skewed chains.
False correlation	Typed/scoped exact keys, conflict checks, explicit missing/ambiguous abstention	An internally consistent wrong assertion can evade structural checks; detector guarantees are workload-specific.
Sensitive evidence	Minimum metadata, source-resident payloads, and separable access	Metadata can remain personal or security-sensitive and requires retention and access policies.
Evidence-path outage	Asynchronous, off-request-path placement	Events may be delayed or dropped unless adapters provide buffering, retries, and monitoring.
Post-run or mutable metadata	Separate the attested TAR from checksum-bound finalization; capture exact tag mapping	Finalized state is not builder-attested, and tag uniqueness is a capture-time observation.

NIST cyber-resiliency guidance emphasizes the ability to anticipate, withstand, recover from, and adapt to adverse conditions [16]. EACP can contribute evidence for those activities, but it must itself be monitored, backed up, access-controlled, and tested. A production design should minimize personal

data, segregate duties, define retention by event class, encrypt transport and storage, and record retrieval of sensitive evidence.

6 Reference Implementation

The published v1.2 microbenchmark uses CPython's standard library and SQLite and can run without cloud accounts or proprietary services. Its fragmented baseline contains six tables with intentionally different field names. Each table is indexed on its native service/time pair and native correlation/time pair. This is stronger than a naive unindexed baseline and reduces the likelihood that EACP appears favorable merely because the alternative was poorly configured. The v1.3 profile tools and offline experiments also use the standard library, whereas the public live protocol necessarily uses GitHub Actions and an ephemeral kind cluster.

The 1.3 profile is implemented as Draft 2020-12 JSON Schemas plus a dependency-free validator, migrator, and safe resolver. The tool rejects duplicate source keys, unscoped or malformed identities, invalid link-method combinations, ambiguous resolution seeds, and unknown fields outside namespaced extensions. A migration test converted all 374 rows of the published Kubernetes projection and revalidated 374/374 records while retaining every legacy field under an auditable extension. This demonstrates backward-compatible data transformation, not semantic enrichment of the old rows.

Table 4. Published v1.2 synthetic source planes and normalized semantics

Source plane	Native service field	Normalized intent	Example pointer scheme
Deployment	app	release_integrity	ci://runs/{id}
Identity	resource_service	least_privilege	iam://events/{id}
Policy	workload	policy_conformance	policy://decisions/{id}
Telemetry	target	service_availability	otel://queries/{id}
Incident	affected_service	incident_response	ticket://incidents/{id}
Recovery	system	recoverability	backup://checks/{id}

The published v1.2 microbenchmark evidence table stores source type and identifier, source and observed time, one actor string, one service string, intent, policy, action, outcome, pointer, one correlation identifier, and a hexadecimal SHA-256 digest. It uses a uniqueness constraint for idempotency; indexes on (service, source_ts), (correlation_id, source_ts), and (source_type, source_ts); and triggers that abort UPDATE and DELETE. Inserts occur in one transaction using 1,000-row executemany batches. No payloads are copied, no network service is invoked, and no application traffic is generated. These flat columns are the backward-compatible 1.2 projection; the normative 1.3 JSON profile uses actor-role maps, scoped service objects, and typed multivalued links.

The same query projection is implemented for both alternatives. The fragmented version performs a UNION ALL across six indexed tables and normalizes native fields in the SELECT clauses. The EACP version uses one indexed table. Both return the same ten fields in the same order. The script compares every measured query result row-for-row and raises an assertion if any value or ordering differs. It also streams the complete ordered projection from each alternative through SHA-256 and rejects the trial if the row count or digest differs. Representative SQLite EXPLAIN QUERY PLAN output is retained with the results to document index use.

A second adapter consumes Kubernetes audit.k8s.io/v1 JSON records. It maps auditID and stage to the source key; request and stage timestamps to source and observed time; the authenticated username to actor; object reference and workload labels to service; verb and HTTP response status to action and outcome; an audit URI to source pointer; and the explicit eacp.io/correlation-id annotation to operational correlation. The auditID identifies one API request and is not presented as a cross-plane chain identifier [23]. The public corpus is namespace-filtered and removes Kubernetes audit sourceIPs fields and local filesystem paths; the complete control-plane audit log remains local QA material.

The GitHub Actions adapter minimizes REST run, job, and artifact metadata into a source snapshot and emits both the legacy 13-column projection and normative 1.3 records. It retains the run and attempt, repository identity, checked-out commit, workflow and job identity, distinct actor roles, source URLs, and declared artifact metadata; it excludes logs, event payloads, artifact contents, commit messages, author email, runner identity, URL query parameters, and credential-like fields. The companion workflow fails closed unless the REST head SHA equals the checked-out revision and exact correlation, OCI-subject, negative-control, and RBAC-denial checks all pass.

For the interoperability and preservation reference exercise, OpenTelemetry Collector Contrib 0.158.0 reads the same frozen JSONL bytes through the filelog receiver and exports OTLP JSON through the file exporter. The Collector retained each raw input line as the exported log body; the configured JSON parser also populated attributes, but those auxiliary attributes were not the input to the preservation check. After export, a shared external validator projects each recovered raw body to the same Kubernetes fields and compares count, field equality, and SHA-256 after stable ordering. The content hash is recomputed by that validator as a preservation check and is not attributed to the Collector; EACP's SQLite indexes have no Collector equivalent. Collector output size and EACP database size are reported descriptively but are not interpreted as like-for-like storage efficiency because the former is an unindexed interchange file and the latter is an indexed evidence store.

7 Experimental Design

7.1 Workload

Three event volumes were evaluated: 10,000, 50,000, and 100,000. Aligned groups of six events form deterministic cross-plane chains—deployment, identity, policy, telemetry, incident, and recovery—with one service and correlation identifier; because none of the requested counts is divisible by six, each dataset ends with one trailing partial group. A coprime modular traversal balances consecutive chains across 200 services. For each trial seed, the smaller datasets are prefixes of the 100,000-event workload. Deterministic predicates introduce occasional failed deployments, access denials, degraded health, investigation states, and rollback outcomes. These variations exercise values and ordering; they are not probabilistic models of real incidents.

For each volume, ten sequential trials use recorded seeds. Each trial measures 300 randomly selected service reconstructions and 300 randomly selected correlation reconstructions, sampled with replacement. Both databases are warmed with representative indexed service queries before timing. Query order alternates between baseline-first and EACP-first to reduce systematic ordering bias. Elapsed time is measured with `perf_counter_ns`; database size is measured after `ANALYZE` and `VACUUM`.

7.2 Adversarial correlation protocol

The robustness campaign creates 600 hidden truth chains per seed, six events per chain, across 24 services. Each chain spans 1,250 ms; 25% of paired same-service chains overlap by beginning 800 ms apart, while the remainder are separated by 10 seconds. Exact hash-ranked selection applies each fault to a declared fraction. Thirty fixed seeds are evaluated under 25 scenarios: complete identifiers; 1%, 5%, 10%, and 20% random omission; the same rates of plausible wrong-ID substitution; loss of each complete source plane; within-service and cross-service reuse; exact replay; same-source conflicting replay; ± 5 -second source-clock skew; 30-second late arrival; 0–3-second jitter; and a compound fault.

Three reconstruction policies are scored without access to hidden truth. The candidate strict policy groups by complete (service, correlation ID), suppresses exact replay, and abstains on missing IDs, source conflicts, duplicate planes, excessive span, or inconsistent six-plane cadence. An ID-only ablation exposes the effect of dropping service scope. A deliberately naive temporal comparator sorts by source time within service and greedily groups inside a 1,500-ms window; it is an illustrative boundary, not a proxy for a SIEM or causal-inference system. Ground truth is used only for pairwise join precision/recall, exact-chain precision/recall/F1, missed and false joins, ambiguity detection, abstention, deduplication, and time-to-completeness scoring. Every trial row is retained.

7.3 Metrics, index ablation, and analysis

Evidence processing is an amortized bulk metric over one transaction and includes tuple normalization from pre-built Python dictionaries, canonical JSON creation, SHA-256 calculation, three evidence-index updates, and one transaction commit. It excludes raw-source insertion, source transport, collector buffering, and source-to-observer delay. Storage is the physical size of each separate SQLite database divided by event count. The EACP size is additional to the fragmented source database; it is not a replacement or savings. Query metrics are per-trial p50 and p95 values. The paper reports the median of ten trial-level values and shows interquartile ranges in Figure 2.

The index ablation reuses the identical v1.2 generator, seeds, databases, selected query keys, and row-equivalence assertions while varying only the evidence-table indexes: both service-time and correlation-time indexes, service-time only, correlation-time only, or neither. Queries are paired and execution order rotates. A warm pass follows representative access; the separate cold-open pass creates a new SQLite connection with no deliberate warm-up, but does not claim eviction from the operating-system page cache. The ablation reports trial-level p95 query latency, query plans, ingest time, database bytes, and complete-projection equivalence.

The benchmark is a deterministic microbenchmark rather than a sample from a defined production population. Accordingly, the analysis is descriptive. Formal hypothesis tests and confidence intervals would suggest a population model that the synthetic trials do not supply. Future production traces and randomized scenario experiments should preregister hypotheses and use paired inference appropriate to the deployment.

7.4 Execution environments

The published v1.2 benchmark executed on an Apple M4 Max host with 16 logical CPUs and 51,539,607,552 bytes of memory, macOS 26.5.1 arm64, CPython 3.11.9, and SQLite 3.51.0. The benchmark is single-process. Both databases use `journal_mode=DELETE`, `synchronous=FULL`, `temp_store=MEMORY`, foreign keys enabled, and a 64 MiB SQLite cache setting. The exact command, environment metadata, trial-level CSV, and summary JSON accompany the manuscript.

The v1.3 correlation campaign and index ablation executed on the same Apple M4 Max class of host under macOS 26.6.2 arm64 and CPython 3.11.9; the ablation used SQLite 3.51.0. Their exact metadata are frozen separately, and the correlation campaign requires no network, containers, or third-party packages. The earlier three-attempt GitHub-to-Kubernetes execution and both nine-run cross-version cohorts used GitHub's ubuntu-24.04 hosted environment, with a fresh ephemeral cluster per execution. The cross-version cohorts held kind 0.32.0, the workload, subject digest, adapter, resolver, controls, and runner class constant while selecting digest-pinned kind node images and checksum-verified matching kubectl binaries for Kubernetes v1.34.8, v1.35.5, or v1.36.1; the job then asserted exact API-server and kubelet versions. Evidence bundles record the exact run, attempt, revision, actions, binaries, container-

image digests, and sanitized source pointers. Environment records describe executions; they are not asserted to be representative production hosts.

7.5 Published Kubernetes corpus and reference-pipeline protocol

The platform exercise used kind 0.32.0 with a single Kubernetes v1.36.1 control-plane node. API-server auditing was enabled through a mounted audit policy [2], [24]. Secrets and authentication/authorization review APIs were excluded, RequestReceived was omitted, and request/response capture was restricted to synthetic non-secret resources in namespace eacp-k8s-eval. Three workload rounds created, labeled, read, listed, and deleted ConfigMaps, patched a Deployment, and issued one unauthorized patch per round through a constrained service account. The driver required each unauthorized request to fail and the audit analyzer required corresponding HTTP 403 records.

After capture, the namespace-filtered corpus was frozen before the preservation exercise. Ten fresh EACP SQLite ingestions and ten fresh OpenTelemetry Collector Contrib 0.158.0 replays [25] consumed identical bytes on the same host. Each replay started with empty output state; completion required the expected event count, a valid serialized output, and projection equality. Elapsed wall time includes pipeline startup and completion detection, so fixed container startup costs can dominate this compact corpus. Trial summaries report median, interquartile range, minimum, and maximum. These sequential same-host replays are descriptive repetitions rather than independent samples from a production population.

7.6 Earlier fixed-version GitHub Actions-to-Kubernetes protocol

A branch-scoped public workflow reads its own run metadata through the GitHub Actions REST API [39], requires the API head SHA to equal the checkout, and derives an attempt-specific key from repository ID, run ID, and run attempt. It installs kind 0.32.0 from a checksum-pinned binary, starts Kubernetes v1.36.1, and deploys registry.k8s.io/pause:3.10 by manifest-list digest. The key and subject digest are written into Deployment and Pod-template annotations. A second Deployment omits the correlation annotation as a negative control. The job then impersonates a read-only service account for a patch against that exact positive Deployment and requires a Forbidden response.

The workflow itself generates the attempt-specific correlation identifier and writes it into the positive Deployment and Pod-template annotations. The experiment therefore evaluates controlled propagation and exact composition of an introduced key; it does not evaluate discovery of naturally occurring identifiers.

The analyzer accepts the positive join only when GitHub and Kubernetes records share the exact scoped correlation value, the declared OCI subject equals the Deployment image, the generated Pod carries the same metadata, the no-ID object remains unjoined, and an HTTP 403 record against the exact declared target and principal exists and is separately classified. It distinguishes the requested image-index digest

from the runtime-reported platform-manifest image ID [41]. The namespace-filtered audit subset is sanitized before upload. The workflow creates a stable archive and SHA-256 manifest, then requests GitHub build-provenance attestation for the archive [40]. Post-run finalization separately captures the now-completed API state and artifact metadata without rewriting the in-run evidence.

7.7 Predeclared cross-version protocol and corrective amendment

Before execution, the plan at commit 15d72da095a0c7640b9318b50b28728e76d68928 fixed a balanced three-version-by-three-repeat cohort: exact tags run-01 through run-03 for each of Kubernetes v1.34.8, v1.35.5, and v1.36.1. Every tag had to point to the same protocol commit, produce a distinct workflow-run ID on attempt one, and be retained whether it succeeded or failed. Tag pushes were issued round-robin by repeat; GitHub queue and start order were observed rather than controlled. Branch pushes were disabled for this evidence workflow. The analysis was predeclared as descriptive and version-stratified, with no pooling with the earlier three-attempt run.

All nine initial first attempts reached the exact kubectl-client, API-server, and kubelet version checks and then failed before artifact upload. The in-job analyzer required three GitHub evidence rows even though the third row describes the artifact created only after that analyzer exits. With no uploaded archive, none of the nine runs satisfied the end-to-end criteria. The nine failed outcomes were retained rather than replaced. Their minimized locally captured run, job, step, and failure excerpts are checksum-bound and cross-checkable against the public GitHub API and logs, but are not themselves origin-authenticated records.

The corrective amendment was committed before confirmatory execution at 4cbf7d2fa0bb44585d258a3f37ce0c0d39ddea43. Its sole scientific acceptance-logic change relocated the artifact-dependent three-row check from in-job execution to completed-state finalization, after upload. The amendment also added the prospectively declared tag allowlist and outcome-capture, summarization, test, and verification support. Those supporting changes did not alter the workload, controls, join semantics, Kubernetes target pins, subject, or scientific acceptance criteria. The confirmatory generation retained the first attempt for each of nine distinct tags.

A successful capture downloads the in-run TAR and its GitHub build-provenance bundle, requires capture-time verification through GitHub CLI's built-in trust configuration, and stores the trusted-root bytes used for offline re-verification. The attestation subject is only the exact in-run TAR. The captured root enables replay relative to captured bytes but is not self-authenticating. Completed-state finalization is generated locally after the workflow ends; it is not builder-attested. It is instead checksum-bound, constrained to the attested runtime run, repository, revision, ref, workflow, artifact name, and correlation fields, and cross-checkable against the public API. Likewise, exact-tag uniqueness is an observation made at capture time, not a proof that a mutable tag can never change later.

PROTOCOL-CORRECTION BOUNDARY

In the initial generation, 0/9 first-attempt runs satisfied all predeclared end-to-end criteria; all nine workflow failures remain preserved. The separately predeclared confirmatory generation evaluates the narrowly corrected lifecycle placement only; it neither reclassifies the failed runs nor estimates a production failure rate. Both generations and every Kubernetes version are reported separately.

8 Results

Table 5. Median evidence-processing and physical-storage results across ten trials

Events	Ingest μ s / event	Events / second	Fragmented bytes / event	EACP bytes / event (multiple; increase)
10,000	4.955	201,839	233.9	414.9 (1.77 $\times$; +77.4%)
50,000	5.062	197,558	227.7	412.8 (1.81 $\times$; +81.3%)
100,000	5.332	187,550	228.4	414.0 (1.81 $\times$; +81.3%)

Amortized bulk processing remained near 5.0–5.3 microseconds per event across the three volumes, or approximately 187,550–201,839 events/s in the measured single-process configuration. This is a local bulk-processing result, not per-event latency or end-to-end ingestion throughput. The physical evidence database consumed approximately 413–415 bytes per event. Because the fragmented source database used about 228–234 bytes per event, the evidence store was 77–81% larger; retaining both produces a combined footprint approximately 2.77–2.81 times the source-only baseline.

Table 6. Median p95 reconstruction-query results across ten trials

Events	Query	Fragmented p95 (ms)	EACP p95 (ms)	Median speedup
10,000	Service state	0.105	0.075	1.38 $\times$
10,000	Correlation chain	0.028	0.019	1.53 $\times$
50,000	Service state	0.510	0.336	1.51 $\times$
50,000	Correlation chain	0.036	0.020	1.80 $\times$
100,000	Service state	1.141	0.691	1.65 $\times$
100,000	Correlation chain	0.045	0.022	2.05 $\times$

Reproducible Pilot Benchmark

Median of 10 sequential seeded trials; whiskers show the interquartile range



Figure 2. Pilot benchmark results. Bars show the median of ten trial-level metrics and whiskers show the interquartile range. The EACP database is an additional evidence index, not a replacement for source storage.

8.1 RQ1: reconstruction adequacy

For the generated workload, the tuple was sufficient to reproduce the measured service and correlation projections exactly. Every one of the 600 sampled query pairs in every trial was compared row-for-row, and all assertions passed. The complete ordered projection also matched in row count and SHA-256 digest for all 30 trials. This result demonstrates schema and query equivalence under complete explicit identifiers. It does not estimate precision or recall for inferred correlation, because the v1.2 prototype performs no inference and that workload contains no missing or ambiguous identifiers.

$$\text{Reconstruction coverage} = \text{correctly reconstructable chains} / \text{total chains}$$

The original experiment therefore estimates reconstruction equivalence conditional on complete, correctly propagated identifiers. Section 8.2 directly tests the previously declared missing, reused, wrong, conflicting, delayed, reordered, and clock-skewed cases under a separate frozen protocol; it does not retroactively change the archived v1.2 result.

8.2 RQ2: degradation and safe-failure behavior

Table 7. Median strict-policy results across 30 deterministic seeds

Fault scenario	Exact-chain accuracy	Abstained observations	False-join rate	Pairwise recall	TTC p95
Control	100.00%	0.00%	0.00%	100.00%	1,275 ms
IDs missing: 1%	94.17%	1.00%	0.00%	98.01%	1,275 ms
IDs missing: 5%	73.25%	5.00%	0.00%	90.23%	1,275 ms
IDs missing: 10%	53.00%	10.00%	0.00%	81.00%	1,275 ms
IDs missing: 20%	26.17%	20.00%	0.00%	64.02%	1,275 ms
Wrong IDs: 20%	7.08%	76.53%	0.00%	18.66%	1,275 ms
Same-service reuse: 10% pairs	80.00%	20.00%	0.00%	80.00%	1,275 ms
Clock skew: 10%, ± 5 s	53.00%	47.00%	0.00%	53.00%	1,275 ms
Late arrival: 10%, +30 s	100.00%	0.00%	0.00%	100.00%	31,275 ms
Compound	47.33%	19.01%	0.00%	72.86%	33,526 ms

Correlation Failure Is a Measured Trade-off

25 scenarios $\times$ 3 policies $\times$ 30 deterministic seeds; medians shown



Figure 3. Correlation robustness under controlled faults. Exact recovery declines nonlinearly for six-event chains; strict mode preserves pairwise precision in these scenarios by making abstention explicit. Medians span 30 deterministic seeds; the shaded policy boundaries are synthetic and workload-specific.

Randomly removing 1%, 5%, 10%, and 20% of event identifiers reduced median exact-chain recovery from 100% to 94.17%, 73.25%, 53.00%, and 26.17%, respectively. This decline is structurally plausible: an exact six-event chain survives only if every required member remains joinable. Losing every identifier from any one plane affected every chain and therefore produced 0% exact-chain recovery, despite retaining 66.67% pairwise recall. Missing evidence was retained individually, not imputed.

Wrong identifiers were more damaging than absent identifiers because one altered event could contaminate a second candidate. At 20% substitution, exact-chain recovery was 7.08% and abstention was 76.53%. In the seeded within-service reuse, wrong-ID, and compound scenarios, the declared duplicate-plane, span, and cadence checks detected every mixed candidate that occurred and strict mode accepted none; pairwise false joins were 0%. This is not a universal no-false-join guarantee: the detector relies on declared synthetic invariants, and a complete internally consistent substitution can evade it.

Service scope mattered. Reusing IDs across services at the 5%-pair rate left strict recovery at 100%, whereas the unscoped ID-only ablation silently accepted 30 ambiguous groups, reduced join precision to 89.29%, and produced a 10.71% false-join rate. The temporal comparator reached 75.33% exact-chain accuracy in the control case but incurred 13.78% false joins. Exact replay of 10% of canonical records was deduplicated without changing recovery; conflicting payload replay was detected for 100% of injected conflicts.

Delay and reordering separated final membership from availability. Strict membership remained exact under isolated 30-second late arrival and 0–3-second jitter, while median p95 time-to-completeness rose from 1,275 ms to 31,275 ms and 4,125.7 ms, respectively. Source-clock skew did not change arrival order but triggered conservative cadence abstention, reducing exact-chain recovery to 53%. Under the compound fault, strict mode produced 47.33% exact recovery, 19.01% abstention, 72.86% pairwise recall, and no observed false join. The result quantifies a safety-availability trade-off rather than proving causal reconstruction.

8.3 RQ3: processing, storage, and query cost

The evidence path added measurable local processing and a substantial storage footprint. Query gains were consistent but modest: service-state p95 speedup increased from 1.38× at 10,000 events to 1.65× at 100,000, and correlation speedup from 1.53× to 2.05×. Average rows returned by a service query increased from approximately 50 to 250 and 500 across the three nested workloads; each correlation query returned six rows. Absolute timings remained below 1.2 ms because both alternatives were local, indexed, and warm-cache. A production environment with separate remote backends could show a larger coordination benefit, a smaller database benefit, or both; this benchmark cannot decide which.

Table 8. Index-ablation medians across ten paired trials

Events	Warm service p95: full → no service index	Warm correlation p95: full → no correlation index	Cold-open service	Cold-open correlation
10,000	0.073 → 0.363 ms (5.08×)	0.023 → 0.370 ms (15.94×)	0.203 → 0.959 ms (4.70×)	0.139 → 0.955 ms (6.99×)
50,000	0.362 → 1.943 ms (5.36×)	0.048 → 2.099 ms (44.62×)	0.546 → 4.513 ms (8.14×)	0.273 → 4.444 ms (15.87×)
100,000	0.691 → 3.884 ms (5.65×)	0.056 → 4.182 ms (73.91×)	0.912 → 8.085 ms (8.84×)	0.295 → 7.886 ms (27.24×)

The ablation resolves an important alternative explanation. Removing only the index targeted by each lookup changed SQLite's plan from an indexed search to a full evidence-table scan at all sizes. At 100,000 events, median warm p95 rose from 0.691 to 3.884 ms for service retrieval and from 0.056 to 4.182 ms for correlation retrieval. The deliberately named cold-open measure—a new connection and local SQLite page cache, without claiming operating-system cache eviction—showed the same direction: 0.912 to 8.085 ms and 0.295 to 7.886 ms. All 18,000 warm and 1,200 cold-open paired query cases remained row-identical across variants.

The performance is therefore an indexed-materialization result, not an automatic consequence of normalization. At 100,000 events, the service and correlation indexes occupied 10.161% and 12.615% of the full 41,398,272-byte database; removing both reduced the file by 22.776% and reduced paired bulk-ingest time by a median 17.90%. All 30 ablation trials passed SQLite integrity and complete-projection checks. The original fragmented alternative was also indexed, so this result exposes the EACP index's cost and necessity without turning the earlier comparison into an indexed-versus-unindexed straw man.

8.4 Cost and index trade-off

The principal trade-off is not 'more evidence for free.' Normalization reduces query fan-out and gives one service-level contract, but retaining the source database and the additional EACP store raises combined physical storage to about 2.8 times the synthetic source-only database in this pilot. The digest and three indexes materially contribute to that cost. Implementers should select fields and indexes from actual query requirements, measure retention tiers, and consider storing binary digests rather than hexadecimal text. The pilot supports feasibility with explicit costs; it does not support a universal claim that EACP is cheaper than querying native tools.

9 Public Cross-Plane Executions, Protocol Correction, and Platform Continuity

9.1 RQ4 baseline: earlier fixed-version public run

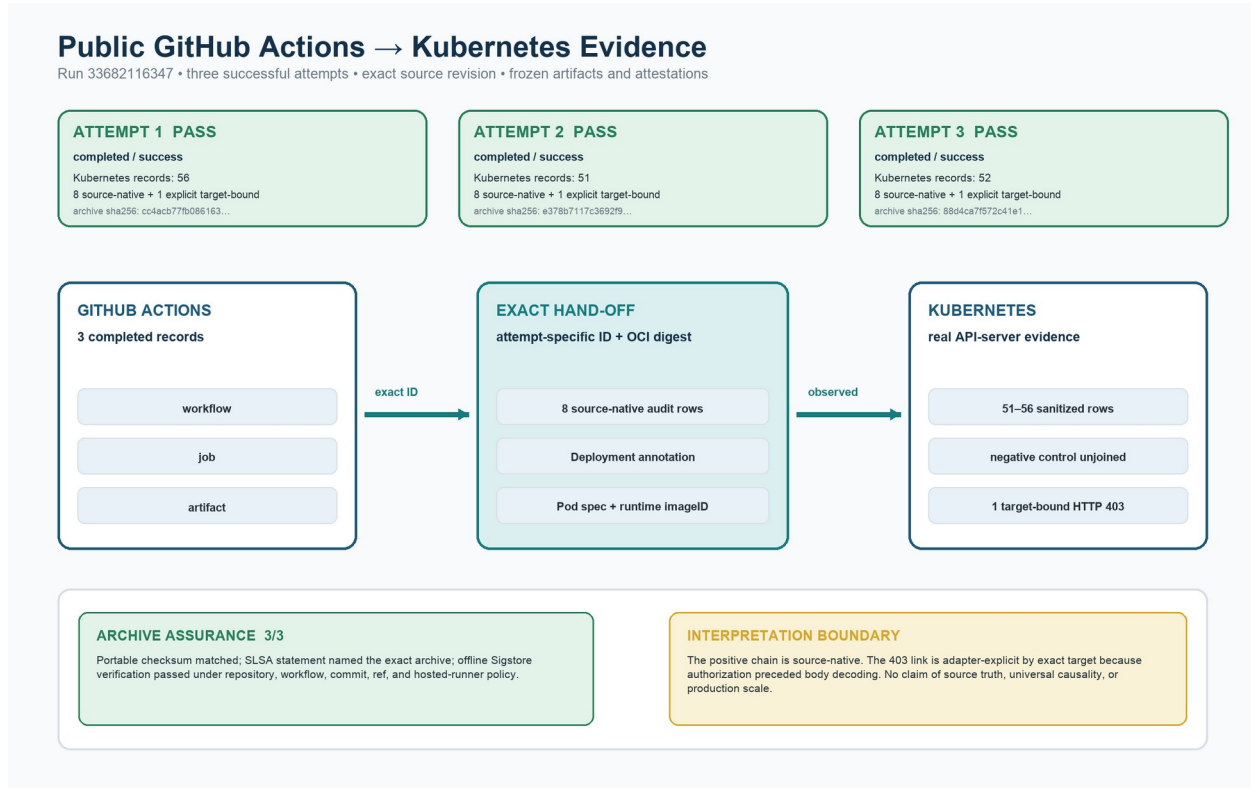


Figure 4. Earlier fixed-version baseline: three attempts of public run 33682116347. The positive GitHub Actions-to-Kubernetes path used an attempt-specific exact identifier and OCI digest; the no-ID control remained unjoined. The HTTP 403 is separately labeled as an adapter-explicit exact-target association.

Table 9. Frozen results from three attempts of public run 33682116347

Attempt	Completed GitHub records	Sanitized Kubernetes records	Source-native positive	Declared controls	Archive assurance
1	3	56	8 audit rows; Deployment + Pod spec/runtime digest exact	3 no-ID records unjoined; 1 target-bound 403	portable SHA-256 + SLSA attestation verified
2	3	51	8 audit rows; Deployment + Pod spec/runtime digest exact	3 no-ID records unjoined; 1 target-bound 403	portable SHA-256 + SLSA attestation verified
3	3	52	8 audit rows; Deployment + Pod spec/runtime digest exact	3 no-ID records unjoined; 1 target-bound 403	portable SHA-256 + SLSA attestation verified

Public GitHub Actions run 33682116347 completed three attempts successfully on 2 September 2026, each against source revision 76b2ed54381ae52cf0f54cd22a20341c3216b77b. Attempt number participates in the correlation key, so the three executions produced distinct identifiers rather than reusing one chain. A post-run read-only recapture closed each GitHub view as completed/success and

produced three profile records—workflow, job, and uploaded artifact—without rewriting the checksum-bound runtime observation.

In every attempt, eight Kubernetes API-server audit records carried the exact source-native correlation annotation. The Deployment, its generated Pod, the Pod specification, and the runtime-reported image ID also matched the declared registry.k8s.io/pause OCI digest exactly. The namespace-filtered corpus contained 56, 51, and 52 unique source records across attempts; this incidental variation reflects asynchronous controller activity and was not used as a success criterion. The final normalized view contained nine records with the operational key: eight source-native records plus the separately classified denied operation.

Here, ‘source-native’ means that the annotation is present in the retained raw Kubernetes audit record; it does not mean that the identifier arose independently of the experiment. The OCI subject-digest match is evaluated as a separate check rather than treated as the operational-correlation join.

The negative control was not a missing observation: its object and three audit records were present in every attempt, but none carried the correlation annotation and none was joined. One patch by the expected read-only service account returned HTTP 403 against the exact apps/deployments/namespace/name tuple of the positive Deployment in every attempt. Kubernetes authorization occurred before the patch body was reliably decoded, so those 403 records carried zero source-native correlation values. The adapter therefore bound them through exact target identity, marked the operational link `evidence_method=explicit`, retained `source_native_correlation_present=false`, and reported that boundary.

Each deterministic in-run archive matched its portable SHA-256 manifest. The downloaded DSSE bundle named that exact archive digest, and `gh attestation verify` accepted all three under the public repository, signer-workflow, source-commit, source-ref, and hosted-runner constraints [40]. The bundles and archives are frozen for offline re-verification. The completed-state recaptures were generated locally after each run and are separately checksum-bound and cross-checkable against the public API; they are not subjects of the builder attestation. The attestation protects builder identity and in-run archive integrity under that trust policy; it does not certify the semantic truth or completeness of GitHub or Kubernetes records.

9.2 RQ4: predeclared cross-version cohorts

Table 10. First-attempt outcomes from separately committed cross-version generations

Generation / tags	Kubernetes	First attempts	Workflow success	All predeclared criteria	Preserved evidence status
Initial / run-01..03	v1.34.8	3	0 / 3	0 / 3	3 failures; no in-run TAR
Initial / run-01..03	v1.35.5	3	0 / 3	0 / 3	3 failures; no in-run TAR

Generation / tags	Kubernetes	First attempts	Workflow success	All predeclared criteria	Preserved evidence status
Initial / run-01..03	v1.36.1	3	0 / 3	0 / 3	3 failures; no in-run TAR
Confirmatory / run-04..06	v1.34.8	3	3 / 3	3 / 3	All criteria satisfied; split trust boundary
Confirmatory / run-04..06	v1.35.5	3	3 / 3	3 / 3	All criteria satisfied; split trust boundary
Confirmatory / run-04..06	v1.36.1	3	3 / 3	3 / 3	All criteria satisfied; split trust boundary

Table 10 treats the initial run-01..03 generation as an observed negative end-to-end result, not discarded setup history. Its aggregate status is failed: all nine first-attempt workflow outcomes are preserved, three per version, and zero produced an in-run TAR or satisfied the predeclared end-to-end criteria. Minimized public-log observations record that all nine reached exact client, API-server, and kubelet version validation before the common lifecycle assertion failed. Because no in-run TAR was uploaded, the aggregate correctly records zero successful correlation, control, digest, TAR-parity, or attestation results; the nine exact version-validation markers remain separately reported as failure-log observations. The common failure point supports the stated harness diagnosis, but does not establish that any initial run would otherwise have passed.

The separately tagged run-04..06 confirmatory generation recorded 9/9 first attempts satisfying all predeclared criteria: 3/3 for Kubernetes v1.34.8, 3/3 for v1.35.5, and 3/3 for v1.36.1. All tags pointed to amendment commit 4cbf7d2fa0bb44585d258a3f37ce0c0d39ddea43 and produced nine distinct workflow-run IDs and nine distinct operational-correlation identifiers. Every verified run matched the requested kubectl-client, API-server, and kubelet version; retained the positive annotation in raw audit evidence; left the present no-ID control unjoined; labeled the exact-target HTTP 403 adapter-explicit with no source-native operational correlation; and matched the Deployment, Pod specification, and runtime image ID to the OCI digest as a separate check.

All nine confirmatory in-run TARs passed nested SHA-256 verification and byte-for-byte attested-TAR parity. Capture-time `gh attestation verify` passed through GitHub CLI's built-in trust configuration under exact repository, signer-workflow, source-commit, source-ref, and hosted-runner restrictions. The captured trusted-root bytes then supported fresh offline re-verification for all nine bundles, but are not self-authenticating. Each attestation names only its in-run TAR as the subject. Each completed-state finalization was created locally after its run, checksum-bound, and bound to the attested runtime fields and public API state, but was not itself builder-attested. The verifier also observed one exact tag mapping per run at capture time; that observation is not a guarantee against future tag mutation.

The two generations are not pooled, and neither is pooled with run 33682116347. The confirmatory result supports controlled compatibility of the amended procedure across three exact Kubernetes minor

versions on one hosted-runner class. Nine successes are descriptive procedural repetitions, not an inferential reliability estimate, independent reproduction, managed-cluster result, or field deployment.

9.3 Published v1.2 Kubernetes continuity result

Table 11. Single-node Kubernetes audit exercise and EACP checks

Measured item	Observed result	Interpretation boundary
Namespace audit corpus	374 events; 373 audit IDs; 374 auditID:stage keys	One request emitted both ResponseStarted and ResponseComplete.
Explicit operational correlation	132 rows tagged eacp-round-01/02/03	Only actions carrying the explicit annotation are counted.
RBAC policy denial	3 HTTP 403 events: 2 preliminary Deployment GETs and 1 ConfigMap PATCH	The workload issued three unauthorized patch commands, but these are the verbs audited as denied.
Persistence integrity	374 stored rows; input count matched; UPDATE and DELETE blocked	SQLite application-boundary controls, not cryptographic immutability.
Local EACP persistence	Median 3.002 ms over 10 trials (8.026 μ s/event)	Small frozen corpus; excludes network and capture delay.

The live API server produced 1,819 total audit records; after excluding a service-account token subresource record, 374 belonged to the evaluation namespace and formed the public, sanitized corpus. Of those, 338 were initiated by kubectl, 132 retained an explicit workload-round correlation annotation, and three represented the expected authorization denials: two preliminary Deployment GETs and one ConfigMap PATCH. EACP stored every auditID:stage key, preserved 374 distinct hashes of sanitized canonical tuples, and returned indexed service and correlation reconstructions. This establishes executable compatibility with one Kubernetes audit configuration; it does not establish cross-plane completeness because the capture covers only the Kubernetes API plane.

9.4 RQ5: interoperability and corpus-preservation reference exercise

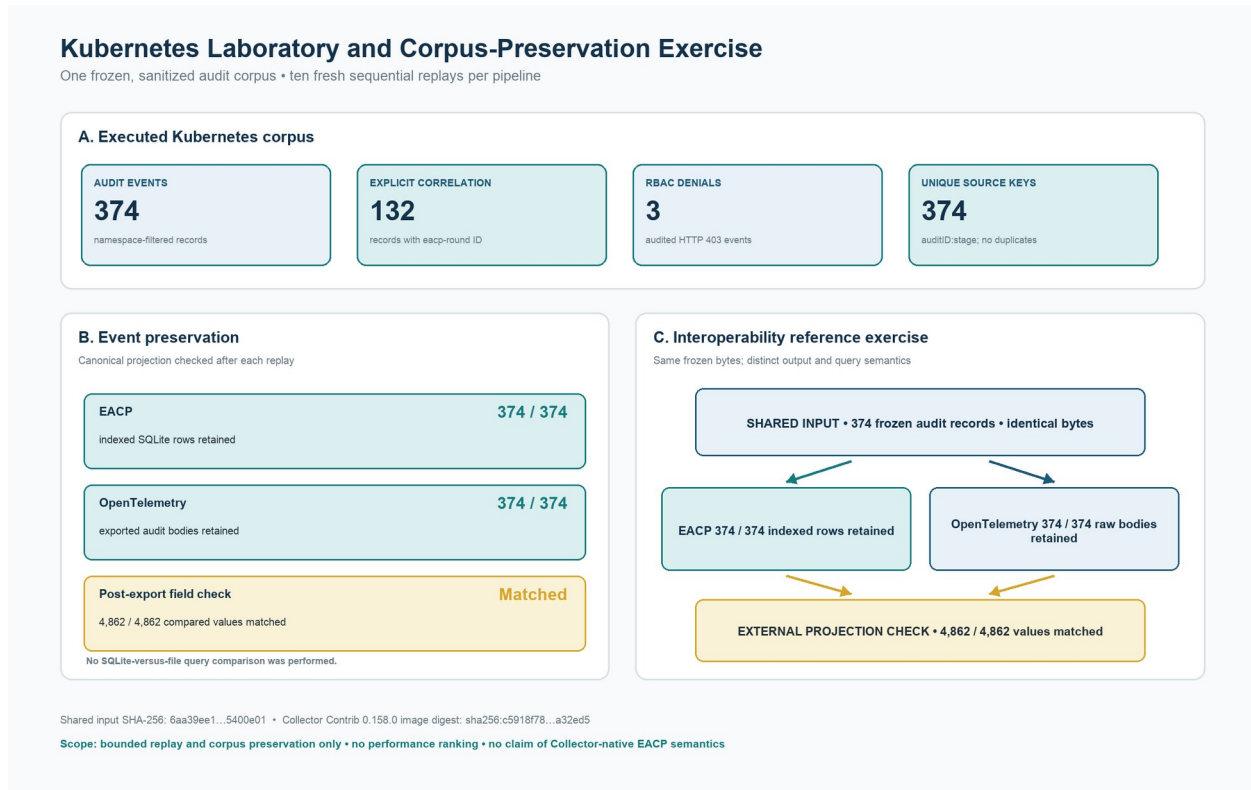


Figure 5. Kubernetes corpus and preservation exercise. Both paths consumed the same frozen bytes and retained all records; the OpenTelemetry path required an external post-export validator for the EACP projection. The figure intentionally presents no performance ranking.

Table 12. Shared-scope preservation results over the same 374-event corpus

Pipeline	Input records	Events retained	Output role	Projection check
EACP	374 frozen audit records	374 / 374 in every run	Indexed SQLite evidence store	4,862 / 4,862 values matched the analyzer reference
OpenTelemetry Collector	374 frozen audit records	374 / 374 raw bodies in every run	OTLP/JSON transport and export	4,862 / 4,862 after external post-export validation

All ten EACP runs and all ten Collector runs retained 374 of 374 events with no duplicate source keys. The Collector retained each raw line as the exported log body; its configured parser also populated attributes, but the post-export EACP validator operated on the retained body. That shared external validator—not the Collector configuration—mapped the recovered audit objects to the 13-field canonical projection. All 4,862/4,862 compared values matched the reference CSV generated separately by the Kubernetes analyzer, and every run produced the same recorded projection digest. This is evidence of content preservation through the configured Collector path; it is not evidence that OpenTelemetry natively generated the EACP semantics.

Figure 5 and Table 12 deliberately restrict the visual comparison to the shared ingest-and-preserve scope. The Collector and EACP have different startup, validation, serialization, storage, and query semantics, so this manuscript does not rank their wall time or byte footprint and does not compare indexed SQLite queries with file scans. The artifact retains the descriptive trial logs for transparency. Within the overlapping scope, both paths retained the complete frozen corpus; only EACP's separate evidence-index function was evaluated for reconstruction queries.

10 Discussion

10.1 Design implications

The central result is not that a shared identifier guarantees a correct history; it is that linkage policy can expose uncertainty instead of concealing it. Exact-chain recovery fell quickly as identifiers disappeared or became wrong, while the strict policy converted observable contradictions into abstention. This separates three properties that are often collapsed: completeness asks whether all expected members were recovered, safety asks whether unrelated members were joined, and availability asks when a defensible chain became queryable. The late-arrival experiment preserved final membership but increased time-to-completeness; the wrong-ID and clock-skew cases instead reduced accepted coverage.

The profile changes are therefore substantive rather than cosmetic. Service identity is scoped, source type and source ID are part of the formal tuple, actor roles are not collapsed into one ambiguous principal, and one record may carry several typed links with explicit evidence methods. The successful earlier and confirmatory executions demonstrate why this vocabulary matters: Kubernetes rows contained the source-native annotation, whereas the denied patch could only be associated through the exact resource target known to the adapter. Reporting both as undifferentiated 'correlated' evidence would erase a material epistemic boundary.

The failed initial cohort also exposes a reproducibility lesson rather than a result to conceal. Completion-dependent metadata cannot be required from a job that has not yet uploaded its artifact. Preserving all nine failures, binding the limited correction to a new commit, and using new first-attempt tags prevents a successful rerun from silently replacing an unfavorable generation. The confirmatory successes validate the amended lifecycle placement under the declared conditions; they do not retroactively turn the initial failures into successes.

The performance evidence also supports a narrower interpretation than 'normalization is faster.' The original comparison placed indexes on both alternatives. The paired ablation then showed that EACP's service and correlation indexes were necessary for its measured lookup latency and accounted for 22.776% of the full database at 100,000 events. Removing both reduced bulk-ingest time by 17.90%. The

practical contribution is a measured retrieval-write-storage trade-off for a materialized index, not a universal efficiency claim.

Cross-plane provenance can consequently be implemented as an index rather than a replacement log lake. Native systems remain authoritative and EACP stores normalized observations, scoped links, and resolvable pointers. That preserves ownership boundaries but makes pointer health, source retention, schema drift, clock semantics, and adapter checkpoints part of the assurance case. The workflow attestations add builder identity and integrity evidence for the in-run TARs only. The locally generated completed-state views have separate checksum and public-API cross-checks; neither mechanism proves that a source event was truthful or converts an observation into causality.

10.2 Deployment path

A prudent deployment would begin with one service and two or three sources, define the service namespace and actor-role mapping, select evidence-relevant event classes, and verify native-pointer retention. Explicit correlation should be propagated through delivery and runtime-control boundaries before any temporal fallback is considered. In observation-only mode, operators can compare each reconstructed chain with native sources and publish four separate objectives: source coverage, accepted-link precision, abstention, and time-to-completeness.

Asynchronous placement is an availability choice. An EACP outage need not block application traffic, but it can create an evidence blind period. Production adapters therefore need durable queues, idempotent writes, retry and dead-letter behavior, source checkpoints, lag objectives, retention compatibility, and alerts for unresolved or multivalued keys. Google SRE practice emphasizes explicit service objectives and error budgets [18]; an evidence path requires the same discipline. Inline enforcement should be a separate design and evaluation because it changes the request-path, threat, and failure models.

The architecture aligns with audit and accountability, configuration management, continuous monitoring, incident response, and contingency-planning outcomes in NIST SP 800-53 [3], but alignment is not certification. An organization must still define controls, assess implementation evidence, meet applicable legal and contractual requirements, and decide whether the evidence index itself falls within regulated scope.

11 Threats to Validity

11.1 Construct validity

The database benchmark measures machine retrieval, not human incident-reconstruction time, audit quality, compliance, resilience, or organizational value. Hidden synthetic truth enables exact linkage scoring, but the six-event chain definition, fault rates, cadence, overlap, and detector invariants are experimental constructs rather than estimates of production behavior. The two-emitter exercise uses

records separately emitted by GitHub Actions and a Kubernetes API server, but both participate in one deliberately orchestrated laboratory operation. The workflow generated and propagated the positive correlation key itself, so this is neither identifier discovery nor independent organizational corroboration. External IAM, telemetry, ticketing, and recovery APIs remain synthetic or absent. 'Evidence' means a retained observation and pointer, not legally admissible proof.

11.2 Internal validity

The fragmented and EACP microbenchmark alternatives use SQLite but different physical schemas. Six UNION branches versus one materialized table is the intended treatment, yet planner and cache behavior affect timings. Warm-up, alternating order, identical pragmas, indexes on both alternatives, ten seeds, and row-equivalence assertions reduce but do not eliminate bias. The index ablation is a separate execution, so its ingest timings should not be substituted for the original run; its cold-open measure resets the SQLite connection-local cache but does not flush the operating-system page cache. Background processes and hardware frequency were not controlled, and no concurrent readers or writers were evaluated.

The correlation generator and scorers share one codebase, although reconstructors cannot access truth-only fields and tests assert that separation. The strict detector's perfect observed mixed-group recall is conditional on its declared duplicate-plane, span, and cadence rules; a plausible complete-chain substitution can evade those checks. The target-bound RBAC link in the live experiment is explicitly adapter-bound through exact Kubernetes resource identity because authorization may precede request-body decoding; it is not relabeled source-native correlation. In the Collector exercise, Python/SQLite, a Go container, batching, serialization, and startup differ, so no cross-pipeline performance ranking is made.

The initial cross-version cohort revealed an experiment-harness defect only after all nine outcomes were observed. The same researcher diagnosed it and implemented the correction. Prospective commitment of the narrow amendment, new tag names observed as distinct at capture time, captured accounting for the 18 selected first attempts, and separate reporting reduce outcome-selection bias, but cannot establish how the initial runs would have ended without the defect. Minimized API metadata and failure-log markers are locally captured and checksum-bound; neither retained capture is an origin-signed response. The completed-state finalizer and attestation-bundle filename normalization also run outside the attested workflow; tests, byte-preserving normalization, checksums, attested-runtime-field binding, built-in GitHub trust verification, and public API cross-checks constrain but do not eliminate that local-capture trust boundary.

11.3 External and conclusion validity

Results from one Apple Silicon host, an earlier three-attempt run, and two nine-run cohorts on ephemeral GitHub-hosted Linux runners, each with one single-node kind cluster, SQLite, and compact controlled corpora cannot be generalized to distributed databases, managed Kubernetes, event buses, remote vendor APIs, production retention, adversarial sources, or high-cardinality services. The synthetic distribution is not calibrated to empirical traces. Trial medians and quartiles describe these deterministic runs only; they are not confidence intervals for a production population. A real deployment may be dominated by network delay, source throttling, clock domains, retention gaps, and operator coordination rather than local query or container startup.

11.4 Researcher and artifact limitations

The architecture, profile, implementation, workload, protocol, failure diagnosis, correction, and interpretation were produced within one project and were not independently replicated or peer reviewed. Public hosted runs, captured accounting for the 18 selected first-attempt outcomes, and capture-time exact-tag observations reduce reliance on the author's workstation and reduce selective-reporting latitude, but do not constitute independent reproduction. Projection equality confirms agreement with declared mappings, not semantic completeness or source truth. Independent reimplementation, blind source-label review, a preregistered temporal-policy sweep, managed multi-node repetitions, and field data are required before stronger claims.

12 Reproducibility and Data Availability

The candidate artifact contains the normative profile and schemas, migration and safe-resolution utility, original synthetic benchmark, adversarial correlation campaign, paired index ablation, public GitHub Actions-to-Kubernetes bundles, the prospective cross-version plan, all nine initial failure captures, the corrective amendment, all nine confirmatory bundles, aggregate run sets and summaries, Kubernetes manifests and analyzer, OpenTelemetry Collector configuration, figure generators, trial-level data, environment metadata, query plans, privacy notes, and layered checksum manifests. The profile and experiment drivers use the Python standard library; figure generation requires Pillow. Live Kubernetes capture additionally requires Docker, kind, kubectl, and GitHub CLI.

Validate the profile implementation and its example record with:

```
python3 -m unittest discover -s spec/tests -p 'test_*.py' -v
python3 spec/tools/eacp_profile.py validate \
spec/examples/valid-record-v1.3.json
```

Reproduce the 30-seed adversarial matrix in a new directory with:

```
python3 experiments/correlation_robustness/correlation_robustness.py \
--chains 600 --services 24 \
--missing-rates 0.01 0.05 0.10 0.20 \
--collision-rates 0.01 0.05 0.10 \
--overlap-fraction 0.25 --temporal-window-ms 1500 \
--output /tmp/eacp-correlation-reproduction
```

Reproduce the paired index ablation without overwriting the frozen reference result with:

```
python3 experiments/index_ablation/index_ablation.py \
--sizes 10 000 50 000 100 000 --trials 10 \
--services 200 --query-samples 300 --cold-open-samples 20 \
--output /tmp/eacp-index-ablation-reproduction
```

The original v1.2 benchmark remains reproducible with:

```
python3 benchmark/sqlite/eacp_benchmark.py \
--sizes 10 000 50 000 100 000 \
--trials 10 \
--services 200 \
--query-samples 300 \
--output /tmp/eacp-v12-reproduction
```

Verify the complete frozen three-attempt live result, then optionally repeat cryptographic verification against each offline attestation bundle:

```
python3 experiments/github_actions/summarize_reference_run.py --verify
# Exact gh attestation commands are in:
# experiments/github_actions/results/reference/run-33 682 116 347/README.md
```

Verify the preserved initial failure generation and the separately captured confirmatory generation with:

```
RESULTS=experiments/github_actions/results/reference
TARGET=experiments/github_actions/kubernetes_targets_v1.3.json
python3 experiments/github_actions/summarize_cross_version_run_set.py \
--root "$RESULTS/cross-version-initial-failed-cohort-v1.3" \
--target-manifest "$TARGET" --verify
python3 experiments/github_actions/summarize_cross_version_run_set.py \
--root "$RESULTS/cross-version-confirmatory-cohort-v1.3" \
--target-manifest "$TARGET" --verify
```

The machine-readable chronology is explicit:
 experiments/github_actions/cross_version_protocol_plan_v1.3.json is the initial prospective plan;
 experiments/github_actions/results/reference/cross-version-initial-failed-cohort-v1.3/
 cross_version_summary.json is its observed result;
 experiments/github_actions/cross_version_protocol_amendment_v1.3.1.json is the prospectively
 committed correction; and experiments/github_actions/results/reference/cross-version-confirmatory-
 cohort-v1.3/cross_version_summary.json is the separate confirmatory result. The prospective status
 fields remain as committed historical state rather than being rewritten after execution.

The published Kubernetes and frozen-corpus reference-exercise entry points remain:

```
./experiments/kubernetes/run_experiment.sh
python3 experiments/comparison/opentelemetry/run_comparison.py run \
--input <public-audit.json> \
--reference-csv <normalized.csv> \
--output-dir <results> \
--trials 10
```

The reviewer candidate is available at

<https://github.com/obedebessa/eacp-operational-provenance/tree/eacp-v1.3-candidate>. The earlier three-attempt evidence snapshot is pinned at <https://github.com/obedebessa/eacp-operational-provenance/tree/c20d2c06efda105cf6772861dd447413c5e709fa>, while the initial cross-version protocol is fixed at <https://github.com/obedebessa/eacp-operational-provenance/tree/15d72da095a0c7640b9318b50b28728e76d68928> and the corrective amendment is fixed at <https://github.com/obedebessa/eacp-operational-provenance/tree/4cbf7d2fa0bb44585d258a3f37ce0c0d39ddea43>. Confirmatory run metadata records that amendment head SHA, and capture-time API queries observed one matching workflow invocation per exact tag; that observation does not guarantee against later tag mutation. The branch retains the initial and confirmatory run sets under `experiments/github_actions/results/reference/`, including every public run URL, minimized failed outcome, successful in-run TAR, completed-state finalization, attestation bundle, verification record, aggregate summary, and layered checksum inventory. The plan, amendment, and observed results remain separate files so prospective declarations are not rewritten after execution. Every declared seed, pinned or recorded relevant version, subject or container digest, privacy rule, trial row, and verification command is recorded next to its result.

Version 1.3 is a reviewer candidate and has no DOI at the time of writing. The prior v1.2 preprint is archived at <https://doi.org/10.5281/zenodo.22017662>; its version-specific software/artifact is archived separately at <https://doi.org/10.5281/zenodo.21818550>, and the Zenodo Concept DOI <https://doi.org/10.5281/zenodo.21817376> identifies the artifact series. None of those identifiers covers this candidate or implies peer review. A future archival release should preserve these raw results, assign a new versioned artifact DOI, and disclose any change to workloads, policies, or interpretation.

13 Declarations

Ethics and data

The study uses deterministic synthetic operational records, one local ephemeral Kubernetes cluster from the published v1.2 exercise, and 21 GitHub-hosted ephemeral clusters populated only with synthetic resources across the earlier three-attempt execution and the two nine-run cohorts. The live evidence also contains public GitHub REST metadata for the author's repository, workflow, jobs, artifacts, and public actor identity. Successful Kubernetes audit corpora were namespace-filtered and sanitized to remove sourceIPs fields, credential identifiers, certificate material, token projections, and local paths; minimized failed-run captures omit raw logs, secrets, and runner identity. Synthetic internal cluster addresses remain only where they are part of resource status. The study includes no human participants, private organizational logs, private repository content, or production credentials.

Funding and competing interests

No external funding was received for this work. The author declares no competing interests.

Copyright and artifact licensing

This manuscript is copyright (C) 2026 Obede Bessa Rocha da Silva, all rights reserved. Licensing in the released version 1.2.0 artifact and this candidate branch is file-scoped: code and configuration are licensed under Apache License 2.0; data, documentation, and figures are licensed under Creative Commons Attribution 4.0 International; and preprint PDFs are expressly excluded from those grants and remain all rights reserved. Any accepted-manuscript or version-of-record license must follow the selected journal's policy and will apply only to the version released under those terms.

AI-use disclosure

OpenAI ChatGPT and Codex (OpenAI; web and desktop interfaces; accessed July–September 2026) were used for literature-discovery support, code scaffolding, benchmark-execution assistance, language drafting, document layout, and editorial organization. The named author independently reviewed the artifact, verified claims and references, and accepts responsibility for the final content. No AI-generated empirical observations were substituted for executed benchmark results; reported values come from the accompanying trial files.

14 Conclusion

EACP 1.3 frames cloud-native governability as a bounded operational-provenance reconstruction problem. Its contribution is a conservative profile and materialized retrieval index: source identity is explicit, service identity is scoped, actor roles remain distinct, records may carry multiple typed links, native evidence pointers are retained, and missing or ambiguous link keys cause abstention. The model composes adjacent mechanisms—including supply-chain attestations, telemetry, audit logs, and provenance systems—without claiming to replace them or to infer causality.

The evaluation replaces the former ideal-identifier assumption with measured failure behavior. Exact six-event recovery fell from 100% to 26.17% as random missingness reached 20%; strict mode converted observable wrong, reused, conflicting, or cadence-inconsistent candidates into abstention under its declared workload invariants. A complete, internally consistent substitution can still evade those checks. A paired ablation isolated the lookup indexes' benefit and cost. An earlier three-attempt public run established controlled delivery-to-Kubernetes composition. In the first prospectively committed three-version cohort, 0/9 first-attempt runs satisfied all predeclared criteria because of a lifecycle-invalid assertion; every failure was retained. After a narrowly scoped amendment on a new commit and new tags, 9/9 confirmatory first attempts satisfied all predeclared criteria, three per Kubernetes version,

while preserving the no-ID, target-bound denial, separate OCI-digest, checksum, TAR-parity, and attestation boundaries.

The defensible conclusion is useful precisely because it is limited. The artifact shows publicly inspectable, procedurally repeated controlled two-emitter composition and compatibility of the amended procedure across three pinned Kubernetes versions, local reconstruction benefit with explicit storage and write costs, detectable uncertainty under controlled faults, transparent preservation of an unsuccessful protocol generation, and continuity with a 374-record Kubernetes/OpenTelemetry preservation exercise. The GitHub build-provenance attestations name the in-run TARs as their subjects; they do not cover the locally finalized completed-state views or establish semantic truth. The study does not establish source truth, universal zero-false-join behavior, tamper-proof evidence, production effectiveness, independent replication, or distributed-scale reliability. The next decisive study is an independently replicated, multi-node or managed-cluster deployment that measures coverage, accepted-link precision, abstention, and evidence delay across delivery, external identity or policy, telemetry, incident, and recovery sources.

References

- [1] Kubernetes, “Controllers,” Kubernetes Documentation, last modified Sept. 1, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>
- [2] Kubernetes, “Auditing,” Kubernetes Documentation, last modified Mar. 16, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>
- [3] Joint Task Force, Security and Privacy Controls for Information Systems and Organizations, NIST SP 800-53 Rev. 5, Release 5.2.0, 2025. doi: 10.6028/NIST.SP.800-53r5. <https://doi.org/10.6028/NIST.SP.800-53r5>
- [4] National Institute of Standards and Technology, The NIST Cybersecurity Framework (CSF) 2.0, Feb. 2024. doi: 10.6028/NIST.CSWP.29. <https://doi.org/10.6028/NIST.CSWP.29>
- [5] Cloud Native Computing Foundation, “CloudEvents—Version 1.0.2,” vendor-neutral event-data specification. [Online]. Available: <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>
- [6] OpenTelemetry, “Logs Data Model,” OpenTelemetry Specification, status: Stable. Accessed Aug. 5, 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/logs/data-model/>
- [7] L. Moreau and P. Missier, eds., PROV-DM: The PROV Data Model, W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [8] T. Lebo, S. Sahoo, and D. McGuinness, eds., PROV-O: The PROV Ontology, W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-o/>
- [9] B. H. Sigelman et al., “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google Research, technical report, 2010. [Online]. Available: <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>
- [10] J. Mace, R. Roelke, and R. Fonseca, “Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems,” in Proc. 25th ACM Symp. Operating Systems Principles, 2015, pp. 378–393. doi: 10.1145/2815400.2815415. <https://doi.org/10.1145/2815400.2815415>
- [11] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, “Provenance-Aware Storage Systems,” in Proc. USENIX Annual Technical Conference, 2006. [Online]. Available: <https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems>
- [12] A. Bates, D. (Jing) Tian, K. R. B. Butler, and T. Moyer, “Trustworthy Whole-System Provenance for the Linux Kernel,” in Proc. 24th USENIX Security Symposium, 2015, pp. 319–334. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>
- [13] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016. doi: 10.1145/2890784. <https://doi.org/10.1145/2890784>
- [14] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004. doi: 10.2307/25148625. <https://doi.org/10.2307/25148625>
- [15] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007. doi: 10.2753/MIS0742-1222240302. <https://doi.org/10.2753/MIS0742-1222240302>
- [16] R. Ross, V. Pillitteri, R. Graubart, D. Bodeau, and R. McQuaid, Developing Cyber-Resilient Systems: A Systems Security Engineering Approach, NIST SP 800-160 Vol. 2 Rev. 1, Dec. 2021. doi: 10.6028/NIST.SP.800-160v2r1. <https://doi.org/10.6028/NIST.SP.800-160v2r1>
- [17] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, Aug. 2015. doi: 10.6028/NIST.FIPS.180-4. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [18] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O’Reilly Media, 2016. [Online]. Available: <https://sre.google/sre-book/table-of-contents/>

- [19] O. B. Rocha da Silva, “Resiliência Digital como Vantagem Institucional: Um Modelo de Governança para Maturidade Cloud e DevOps em Instituições Públicas, Educacionais e Comunitárias,” *Revista CIA—Publicações Científicas (Ciclo do Conhecimento)*, July 21, 2026. Accessed Aug. 6, 2026. [Online]. Available: <https://centraldeinteligenciaacademica.blogspot.com/2026/07/resiliencia-digital-como-vantagem.html>
- [20] OpenTelemetry, “Collector,” OpenTelemetry Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://opentelemetry.io/docs/collector/>
- [21] OpenTelemetry Collector Contrib, “File Log Receiver,” version 0.158.0. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/v0.158.0/receiver/filelogreceiver/README.md>
- [22] OpenTelemetry Collector Contrib, “File Exporter,” version 0.158.0. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/v0.158.0/exporter/fileexporter/README.md>
- [23] Kubernetes, “Audit Configuration API (audit.k8s.io/v1),” Kubernetes Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/config-api/apiserver-audit.v1/>
- [24] Kubernetes SIGs, “Auditing with kind,” kind Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://kind.sigs.k8s.io/docs/user/auditing/>
- [25] OpenTelemetry, “OpenTelemetry Collector Contrib 0.158.0,” release notes, Aug. 4, 2026. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-releases/releases/tag/v0.158.0>
- [26] K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST SP 800-92, Sept. 2006. doi: 10.6028/NIST.SP.800-92. <https://doi.org/10.6028/NIST.SP.800-92>
- [27] in-toto Project, “in-toto Specification,” version 1.0.0, June 2, 2023. [Online]. Available: <https://github.com/in-toto/specification/blob/v1.0.0/in-toto-spec.md>
- [28] S. Torres-Arias et al., “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes,” in *Proc. 28th USENIX Security Symposium*, 2019, pp. 1393–1410. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [29] SLSA Community, “SLSA Specification,” version 1.2, approved Nov. 24, 2025. [Online]. Available: <https://slsa.dev/spec/v1.2/>
- [30] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software Signing for Everybody,” in *Proc. 2022 ACM SIGSAC Conf. Computer and Communications Security*, pp. 2353–2367. doi: 10.1145/3548606.3560596. <https://doi.org/10.1145/3548606.3560596>
- [31] Tekton Project, “Tekton Chains: Supply Chain Security,” version 0.29.0, Aug. 4, 2026. [Online]. Available: <https://github.com/tektoncd/chains/tree/v0.29.0>
- [32] Grafeas Project, “Grafeas: A Component Metadata API,” version 0.2.3, Aug. 28, 2023. [Online]. Available: <https://github.com/grafeas/grafeas/releases/tag/v0.2.3>
- [33] OpenTelemetry Authors, “Semantic Conventions for CI/CD,” semantic-conventions version 1.44.0, Release Candidate, Aug. 4, 2026. [Online]. Available: <https://github.com/open-telemetry/semantic-conventions/blob/v1.44.0/docs/cicd/README.md>
- [34] A. Gehani and D. Tariq, “SPADE: Support for Provenance Auditing in Distributed Environments,” in *Middleware 2012, LNCS 7662*, pp. 101–120. doi: 10.1007/978-3-642-35170-9_6. https://doi.org/10.1007/978-3-642-35170-9_6
- [35] T. Pasquier et al., “Practical Whole-System Provenance Capture,” in *Proc. 2017 ACM Symp. Cloud Computing*, pp. 405–418. doi: 10.1145/3127479.3129249. <https://doi.org/10.1145/3127479.3129249>
- [36] D. Kulkarni, “Provenance Issues in Platform-as-a-Service Model of Cloud Computing,” in *Proc. 7th USENIX Workshop on Hot Topics in Cloud Computing*, 2015. [Online]. Available: <https://www.usenix.org/conference/hotcloud15/workshop-program/presentation/kulkarni>
- [37] T. Li et al., “ProvenanceLens: Service Provenance Management in the Cloud,” in *Proc. CollaborateCom 2014*. doi: 10.4108/icst.collaboratecom.2014.257339. <https://doi.org/10.4108/icst.collaboratecom.2014.257339>
- [38] cloud-ark, “kubeprovenance: Kubernetes Aggregated API Server to Track Lineage/Provenance of Kubernetes Custom Resources,” software repository, 2018–2019. Accessed Sept. 2, 2026. [Online]. Available: <https://github.com/cloud-ark/kubeprovenance/tree/6f5f0be39da6d204dcf4e953dcb8559f594bebfd>

- [39] GitHub, "REST API Endpoints for Workflow Runs," GitHub Docs. Accessed Sept. 2, 2026. [Online]. Available: <https://docs.github.com/en/rest/actions/workflow-runs>
- [40] GitHub, "Using Artifact Attestations to Establish Provenance for Builds," GitHub Docs. Accessed Sept. 2, 2026. [Online]. Available: <https://docs.github.com/en/actions/how-tos/secure-your-work/use-artifact-attestations/use-artifact-attestations>
- [41] Open Container Initiative, "OCI Image Manifest Specification," image-spec version 1.1.1. [Online]. Available: <https://github.com/opencontainers/image-spec/blob/v1.1.1/manifest.md>
- [42] in-toto Project, "Predicate Type: Runtime Trace," in-toto Attestation Framework, predicate version 0.1.0, release 1.2.0, Mar. 18, 2026. [Online]. Available: <https://github.com/in-toto/attestation/blob/v1.2.0/spec/predicates/runtime-trace.md>